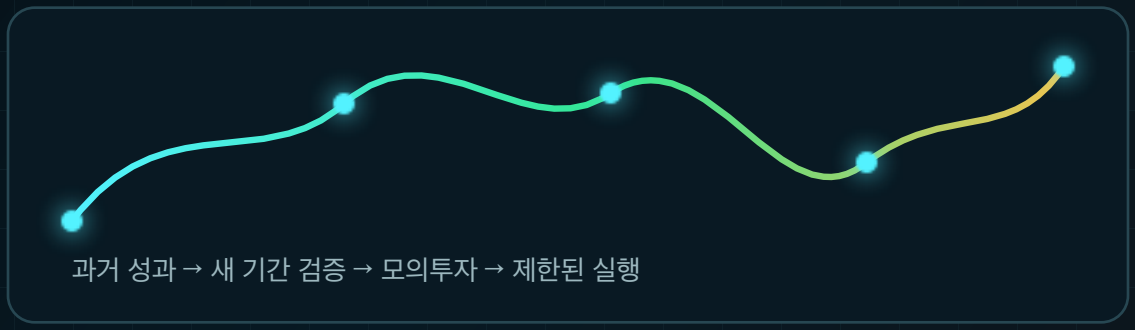
AI QUANT TRADING WITH PYTHON

Implementing AI Quant Trading with Python

HMM·칼만 필터·통계적 차익거래부터 강화학습·백테스트·위험관리까지



Ted Park

데이터 | 검증 | 안전장치 | 실행

Implementing AI Quant Trading with Python

HMM·칼만 필터·통계적 차익거래부터 강화학습·백테스트·위험관리까지

Copyright © 2026 Ted Park.

All rights reserved.

First edition, 2026.

Set in Apple SD Gothic Neo | Menlo.

차 례

머리말 **1**

본문 코드의 범위와 실습 방법 . . . 1

이 장의 흐름 . . . 1

학습 목표 . . . 2

나를 속인 차트 . . . 2

기술적 분석의 현실 . . . 3

르네상스 테크놀로지스 사례에서 얻은 관점 . . . 3

시장을 바라보는 다른 방식 . . . 3

AI 코딩 어시스턴트가 가능하게 한 것 . . . 4

이 책을 마치면 만들 수 있는 것 . . . 4

이 책을 머신러닝 트레이딩 학습서로 읽는 법 . . . 5

읽다가 길을 잃었을 때 보는 지도 . . . 10

하루를 따라가 보는 전체 예시 . . . 11

모델을 더 쉽게 쪼개면 . . . 13

르네상스 사례에서 얻은 원칙 . . . 13

1. 인과성 원칙 . . . 13

2. 앙상블 원칙 . . . 14

3. Walk-forward 검증 원칙 . . . 14

앞으로의 여정 . . . 14

참고: 백테스트 성과를 읽는 법 . . . 15

전체 챕터 로드맵 . . . 15

회독별로 다르게 읽기 . . . 16

Part별 최소 이해 목표 . . . 17

AI를 페어 프로그래머로 쓰기 . . . 18

이 장에서 반드시 기억할 것 19
사전 요구사항 19

0장. 바이브코딩 퀵스타트—먼저 한 번 검증해보기 20

먼저 답을 정한다 20
본문만으로 실행하는 확인 코드 21
AI에게 요청하는 방법 22
시간 순서를 확인한다 22
실패를 좁히는 순서 22
다음 장으로 넘어갈 기준 23

머리말

이 책은 **본문의 설명·계산 예·상태 흐름으로 설계와 검증 원리를 이해하도록** 구성했다. 별도의 소스 코드 파일, 예제 ZIP 또는 저장소 접근권한은 제공하지 않는다. `app.trading`이나 상용 앱 클래스 이름이 등장하는 코드는 시스템 구조를 설명하는 발췌이며, 그대로 실행할 수 있는 전체 프로그램을 뜻하지 않는다.

본문 코드의 범위와 실습 방법

각 장의 독립 실행 예제는 본문 코드를 직접 파일로 저장해 실행한다. 필요한 실행 조건과 기대 결과는 해당 절에 표시했다. 나머지 구조 설명용 코드는 입력·계산 과정·검증 기준을 이해하기 위한 예시이며, 전체 상용 앱이나 자동매매 운영 시스템의 소스를 제공하는 것은 아니다.

이 장의 흐름

이 머리말은 책의 방향을 먼저 잡는다. 차트 중심의 판단이 왜 한계에 부딪히는지 보고, 이 책이 어떤 방식으로 퀀트 시스템을 만드는지 확인한다.

> **TIP**
>
> **가장 빠른 시작**: 앞의 3쪽에서 이 책이 자신에게 맞는지 확인했다면, 이 머리말을 한 번에 완독하지 않아도 된다. 0장으로 이동해 첫 실행을 마친 뒤 돌아와도 흐름이 끊기지 않는다. 뒤쪽 원칙·로드맵·AI 활용법은 막힐 때 찾아보는 안내서로 써도 된다.

- 차트가 나를 속였던 경험
- 기술적 분석이 지금 시장에서 흔들리는 이유
- 르네상스 테크놀로지스 사례에서 가져온 핵심 관점
- 이 책의 모델들이 하나의 시스템으로 연결되는 방식
- 앞으로 만들 시스템과 필요한 준비

학습 목표

이 머리말을 읽고 나면 다음을 설명할 수 있어야 한다.

- 왜 이 책이 차트 패턴이 아니라 데이터 파이프라인과 검증에서 시작하는지
- “미래 데이터를 몰래 참고하는 오류(Look-ahead Bias)”가 왜 퀀트 시스템에서 치명적인지
- HMM, 강화학습, 통계적 차익거래가 이 책 안에서 어떤 역할을 맡는지
- 학습, 추론, 피처, 결측치 같은 머신러닝 기본 용어가 트레이딩 시스템에서 무엇을 뜻하는지
- 이 책을 읽기 전에 어느 정도의 Python과 통계 지식이 필요한지

나를 속인 차트

캔들스틱 차트를 한참 들여다보다가 매수 타이밍이라고 판단한 적이 있다. 같은 지지선에서 세 번이나 반등했고, RSI는 과매도권에 있었으며, MACD 골든크로스도 교과서에서 보던 모양 그대로였다. 여러 신호가 한 방향을 가리키는 것처럼 보였고, 나는 그 신호를 믿고 매수했다.

다음 날 가격은 그“완벽한”지지선을 그대로 뚫고 내려갔다. 처음 겪은 일도 아니었고, 마지막도 아니었다. 다만 그날 이후로 질문이 조금 달라졌다. 단순히 내가 차트를 못 읽은 것인지, 아니면 애초에 내가 의지하던 도구들이 지금의 시장을 설명하기에 부족한 것인지 따져봐야 했다.

기술적 분석의 현실

그 후 몇 달 동안 논문, 트레이딩 포럼, 성과 데이터를 찾아보며 같은 질문을 반복했다. 결론부터 말하면, 내가 의지하던 도구들은 지금의 시장을 설명하기에 부족했다.

전통적인 기술적 분석이란 캔들스틱 패턴, 이동평균 크로스오버, 볼린저밴드, RSI 같은 도구들을 말한다. 이런 도구가 쓸모없다는 뜻은 아니다. 다만 패턴 하나만으로 안정적인 우위를 주장하려면 시장 국면, 거래 비용, 표본 밖 성과까지 함께 검증해야 한다. 내가 쓰던 방식에는 그 검증이 빠져 있었다.

오늘날 시장에서는 알고리즘이 사람보다 훨씬 빠르게 주문을 처리하고, 거래량의 큰 비중을 알고리즘이 차지한다. 과거에는 군중 심리를 반영하던 패턴이, 이제는 기계가 먼저 감지하고 먼저 거래하는 신호가 됐다.

차트에서 지지선이 보이면 사람만 아는 게 아니다. 자동화된 전략도 같은 가격대의 주문 흐름과 유동성을 관찰한다. 스톱로스 주문이 몰린 구간에서는 체결과 변동성이 커질 수 있다. 따라서 과거에 관찰한 패턴이 지금도 같은 방식으로 작동한다고 가정해서는 안 된다.

비관적인 얘기를 하려는 것이 아니다. 개인 투자자가 흔히 쓰는 도구 중 상당수가 지금과는 다른 시장을 전제로 만들어졌다는 사실을 짚는 것뿐이다.

그래서 이 책은“차트를 더 잘 읽는 법”에서 시작하지 않는다. 대신 시장을 데이터를 만들어내는 하나의 과정으로 본다. 그 안에 숨은 상태를 추정하고, 그 추정이 미래 데이터를 몰래 참조하지 않았는지 검증하는 방향으로 나아간다.

르네상스 테크놀로지스 사례에서 얻은 관점

이런 접근을 가장 먼저, 그리고 가장 철저하게 실천한 곳이 르네상스 테크놀로지스다.

짐 사이먼스는 1978년 모네메트릭스(Monemetrics)를 설립했고, 회사는 1982년 르네상스 테크놀로지스로 이름을 바꿨다. 메달리온 펀드는 1988년에 출범했다. 르네상스는 전통적인 재량 트레이더보다 수학자, 물리학자, 컴퓨터 과학자를 적극적으로 채용해 가격 데이터의 통계적 패턴을 연구한 것으로 알려져 있다. 메달리온은 이후 이례적으로 높은 장기 성과를 기록했다.

시장을 바라보는 다른 방식

사이먼스가 특별한 직관을 타고난 것은 아니다. 차이는 시장을 바라보는 방식에 있었다.

이 책은 르네상스의 공개된 정량적 접근에서 한 가지 관점을 가져온다. 시장을 하나의 고정된 법칙으로 설명하기보다, 성격이 다른 여러 국면이 바뀌는 과정으로 보는 것이다. 이것이 르네상스가 HMM을 썼다는 주장은 아니다. 구체적인 내부 모델은 공개되지 않았다. 이 책에서는 그 관점을 구현할 도구로"숨겨진 상태(hidden state)"를 추정하는 HMM을 선택한다.

이 상태들은 눈에 보이지 않지만 데이터로 추정할 수 있다. 현재 시장 상태를 파악하면 그에 맞는 전략도 고를 수 있다.

HMM(은닉 마르코프 모델, Hidden Markov Model)은 바로 이"보이지 않는 상태를 데이터로 추정하는"문제를 다루는 도구다. 이 책 중반부에서 자세히 다룬다.

> 나는 르네상스 테크놀로지스 내부자가 아니고, 그 회사의 정확한 방법론을 알지 못한다. 다만 공개된 인터뷰와 학술 연구를 참고해 그 철학의 일부를 코드로 구현해볼 수 있었다. 이 책은 그 과정을 따라가는 기록이다.

AI 코딩 어시스턴트가 가능하게 한 것

2023년 이후 AI 코딩 어시스턴트가 실용적인 수준에 도달했다. 그 덕분에 이 책을 쓸 수 있었다. GitHub Copilot, Claude, ChatGPT 같은 도구는 복잡한 알고리즘을 설명하고, 미묘한 구현 오류를 찾고, 수식이 코드에서 어떤 의미인지 풀어주는 데 도움이 된다. 이해를 대신해주지는 않는다. 하지만 이해에 도달하는 시간과 시행착오는 크게 줄여준다.

나는 이 책의 시스템을 AI와 함께 만들었다. 어려운 수학에서 막힐 때, 미묘한 버그를 찾을 때, 하이퍼파라미터(모델의 성능을 좌우하는 설정값)를 정할 때 모두 AI의 도움을 받았다. 이 책은 그 과정을 처음부터 다시 밟는다. 이론 설명에서 멈추지 않고, 지금 쓸 수 있는 도구로 실제 시스템을 어떻게 만드는지 보여주는 것이 목표다.

이 책을 마치면 만들 수 있는 것

시스템	한 줄 설명
HMM 시장 국면 감지기	미래 데이터 없이, 지금까지의 데이터만으로 시장이 어떤 국면(상승·횡보·하락)에 있는지 추정한다.
여러 자산을 함께 보는 HMM	주식, 채권 등 여러 자산의 정보를 함께 읽고 단일 자산 HMM에 보완 정보가 되는지 검증한다.
2단계 거시/미시 HMM	거시 경제와 개별 자산을 두 단계로 보고 시장을 Risk-On(위험 선호), Neutral(중립), Risk-Off(안전 선호)로 분류한다.
강화학습 트레이딩 에이전트	HMM이 감지한 국면을 입력받아 포지션을 스스로 결정한다.
통계적 차익거래 시스템	함께 움직이는 자산 쌍을 찾아 그 가격 차이를 노린다.
실전 배포 시스템	IBKR(Interactive Brokers)에 연결해 주문을 내고 체결 상태를 확인한다.

각 부품이 왜 필요한지, 잘못 만들면 어떻게 무너지는지를 이해하는 것이 이 책의 진짜 목표다.

이 책을 머신러닝 트레이딩 학습서로 읽는 법

이 책은 HMM과 강화학습을 다루지만, 더 넓게 보면 **머신러닝으로 주식 트레이딩 시스템을 만드는 학습서**다. 그래서 처음부터 몇 가지 말을 분명히 해두는 편이 좋다.

조금 더 정확히 말하면, 이 책은 **머신러닝과 딥러닝으로 주식시장을 파헤치는 책**이다. 차트를 눈으로 보며"오를 것 같다"를 말하는 대신, 시장을 숫자로 쪼개고, 그 숫자에서 반복되는 구조를 찾고, 그 구조가 실제 매매에서 통하는지 검증한다.

전체 흐름은 다음 한 줄이다.

> `주식시장 -> 데이터 -> 피처 -> 머신러닝 모델 -> 딥러닝 에이전트 -> 검증 -> 주문`

처음 읽을 때는 이 한 줄만 기억해도 충분하다.

단계	쉬운 말	이 책에서 하는 일
주식시장	복잡하게 움직이는 대상	가격, 거래량, 변동성, 금리, VIX가 계속 바뀐다
데이터	시장을 기록한 숫자	OHLCV, 스프레드, 매크로 지표를 모은다
피처	모델이 읽기 쉬운 요약	수익률, 변동성, 모멘텀, Z-score로 바꾼다
머신러닝	시장 상태를 읽는 도구	HMM으로 레짐을 추정한다
딥러닝/RL	행동을 배우는 도구	SAC가 포지션 크기를 정한다
검증	속았는지 확인하는 과정	OOS와 Walk-forward로 성과를 본다
주문	실제 돈이 움직이는 단계	리스크 조건을 통과한 신호만 보낸다

중요한 점은"예측"보다"분해"다. 이 책은 내일 종가 하나를 맞히려는 책이 아니다. 시장을 여러 조각으로 나누어 본다. 지금이 평균회귀(가격이 평균 수준으로 되돌아오는 성질)가 먹히는 장인지, 추세가 강한 장인지, 위험해서 쉬어야 하는 장인지를 차례로 묻고, 그에 맞춰 포지션을 얼마나 가져갈지 정한다.

```
질문 1: 지금 시장은 어떤 상태인가?      -> HMM
질문 2: 이 페어는 벌어졌는가?           -> Z-score / StatArb
질문 3: 지금 얼마나 크게 들어갈까?      -> SAC 강화학습
질문 4: 이 결과를 믿어도 되는가?        -> OOS / Walk-forward 검증
```

이렇게 보면 뒤에서 나오는 HMM, 칼만 필터, SAC, MLflow가 따로 노는 기술이 아니다. 모두 하나의 목적을 위해 이어진다. **시장을 숫자로 읽고, 모델로 판단하고, 검증으로 걸러낸 뒤, 안전하게 실행하는 것**이다.

그렇다면 이 흐름의 한가운데에서 판단을 맡는 모델이란 무엇일까. 머신러닝 모델은 마법처럼 가격을 맞히는 장치가 아니다. 과거 데이터에서 반복되는 구조를 찾아, 새 데이터가 들어왔을 때 현재 상황을 판단하는 도구다.

여기서 **모델(model)**이라는 말이 처음에는 헷갈릴 수 있다. 직관적으로는 이렇게 생각하면 된다.

> **모델 = 숫자를 넣으면 판단을 돌려주는 계산기**

일반 계산기는 2 + 3을 넣으면 5를 돌려준다. 머신러닝 모델은 오늘의 수익률, 변동성, 거래량 같은 숫자를 넣으면"평균회귀 70%, 추세 20%, 위험 10%"같은 판단을 돌려준다.

```
오늘 시장 숫자들  --->  학습된 모델  --->  현재 시장 판단
수익률, 변동성           HMM               MR 70%, Trend 20%, Volatile 10%
```

Python 함수에 비유하면 아래와 같다.

```
def model(today_features):
    return prediction
```

함수처럼 입력을 넣으면 출력이 나온다. 차이는 함수 안의 규칙을 사람이 전부 직접 쓰지 않는다는 점이다. 머신러닝에서는 알고리즘이 과거 데이터를 보고 함수 안의 숫자들을 맞춘다. 학습이 끝난 뒤에는 새 피처를 넣어 판단을 얻는다.

이 관계를 가장 짧게 쓰면 다음 세 줄이다.

```
데이터로 학습한다        -> 모델 안의 숫자가 정해진다
새 피처를 넣는다         -> 모델이 판단을 낸다
그 판단을 검증한다       -> 실전에 쓸지 말지 결정한다
```

실제 시스템에서는 학습이 끝난 모델을 파일로 저장해 둔다. 그래야 매번 처음부터 다시 학습하지 않고, 필요할 때 파일을 불러와 바로 추론할 수 있다.

```
학습 단계
5년치 데이터 -> 피처 생성 -> model.fit(X_train) -> 모델 파일 저장
                                          예: hmm_model.pkl, sac_actor.pt

추론/서빙 단계
오늘 데이터 -> 피처 생성 -> 모델 파일 로드 -> model.predict(today_features) ->
판단 결과
```

여기서 **서빙(serving)**은 저장된 모델 파일을 메모리에 올려두고, 다른 코드가"오늘 피처를 줄 테니 판단 결과를 달라"고 요청할 수 있게 열어두는 일이다. 작은 프로젝트에서는 Python 코드가 그 파일을 직접 불러와도 된다. 운영 시스템에서는 FastAPI 같은 서버가 이를 미리 올려 두고 HTTP 요청으로 결과를 돌려준다.

헷갈리는 말	쉬운 설명	이 책의 예
데이터	모델이 공부하는 재료	5년치 OHLCV, 거래량, 금리
피처	데이터를 모델이 읽기 좋게 바꾼 숫자	수익률, 변동성, 모멘텀
알고리즘	모델을 만드는 방법	HMM의 Baum-Welch, SAC 학습 알고리즘
파라미터	학습으로 정해지는 함수 안의 숫자	HMM의 A, B, π / 신경망 가중치
모델	학습된 파라미터를 가진 판단 함수	오늘 피처 -> 시장 상태 확률

요리로 비유하면 데이터는 재료, 알고리즘은 조리법, 파라미터는 불 세기와 양념 비율, 모델은 그 조리법과 비율이 정해진 완성된 조리 방식이다. 같은 조리법이라도 어떤 재료로 연습했는지에 따라 결과가 달라진다. 머신러닝도 다르지 않다. 같은 HMM 알고리즘이라도 어떤 기간의 데이터를 학습했는지에 따라 만들어지는 모델은 달라진다.

이 책의 HMM도 마찬가지다. HMM은 내일 종가를 직접 예측하지 않는다. 과거의 수익률, 변동성, 거래량 패턴을 보고"지금 시장이 평균회귀에 가까운가, 추세에 가까운가, 위험 구간에 가까운가"를 확률로 추론한다. 이렇게 구분한 시장 상태를 앞으로 **레짐(regime)**이라고 부른다.

앞으로 반복해서 나올 머신러닝 용어를 트레이딩 언어로 바꾸면 다음과 같다.

용어	일반적인 뜻	이 책에서의 뜻
데이터	모델이 읽는 원재료	OHLCV, 스프레드, 금리, VIX 같은 시장 기록
피처(feature)	원자료를 모델이 읽기 쉽게 바꾼 숫자	수익률, 변동성, 모멘텀, 거래량 비율, Z-score
학습(training / fit)	과거 데이터로 모델 파라미터를 맞추는 일	HMM의 레짐 분포, SAC의 정책 네트워크를 과거 구간에서 조정
추론(inference / predict)	학습된 모델로 새 입력을 판단하는 일	오늘까지의 피처로 현재 레짐 확률이나 포지션 크기를 계산
결측치(missing value / NaN)	값이 비어 있거나 계산할 수 없는 칸	상장 전 데이터, 거래 정지, rolling window 초기 구간

용어	일반적인 뜻	이 책에서의 뜻
검증(validation / OOS)	처음 보는 데이터에서도 통하는지 확인	훈련에 쓰지 않은 뒤쪽 기간으로 백테스트

이 구분이 중요한 이유는 간단하다. **학습과 추론을 섞으면 백테스트가 거짓말을 한다.** 학습은 과거 훈련 구간에서만 해야 하고, 추론은 그 시점까지 이미 관측된 데이터만 써야 한다. 결측치를 아무렇게나 0으로 채워서도 안 된다. 0은"값이 없다"가 아니라"정말 0이다"라는 뜻으로 모델에 전달되기 때문이다.

가장 쉬운 예로 5년치 데이터가 있다고 하자. 앞 2년은 모델이 공부하는 구간이고, 뒤 3년은 모델이 처음 보는 시험 구간이다. 실제 비율은 전략마다 달라질 수 있지만, 개념은 항상 같다.

```
전체 데이터 5년

과거                                                    최근
|--------------------|----------------------------------------|
       앞 2년                          뒤 3년
       학습 데이터                     OOS / 추론 데이터
       model.fit()                       model.predict(), step_forward()

모델이 여기서 규칙을 배움          여기서는 규칙을 고치지 않고 적용만 함
```

여기서 뒤 3년을 **OOS(Out-of-Sample)** 라고 부른다. 말 그대로"샘플 밖", 즉 학습에 쓰지 않은 구간이다. OOS는 과거 데이터 안에 있지만, 모델 입장에서는 미래처럼 남겨둔 시험지다. 앞 2년으로 학습한 모델이 뒤 3년에서도 버티면, 그제야"과거를 외운 것만은 아닐 수 있다"고 말할 수 있다.

반대로 5년 전체로 먼저 학습한 뒤 같은 5년에서 성과를 재는 것은, 시험지를 미리 보고 시험을 치는 일과 같다. 수익률은 좋아 보이지만 실전에서는 재현되지 않는다.

따라서 이 책을 읽을 때는 매 장에서 다음 네 질문을 스스로 던져 보면 된다.

1. 지금 모델이 읽는 피처는 무엇인가?
2. 이 피처에 결측치나 미래 정보가 섞이지 않았는가?
3. 학습은 어느 기간으로 했는가?
4. 추론은 실제 거래 시점에도 같은 방식으로 재현 가능한가?

HMM, 칼만 필터, SAC, MLflow 같은 이름은 뒤에서 하나씩 익히면 된다. 먼저 잡아야 할 뼈대는 **데이터 -> 피처 -> 학습 -> 추론 -> 검증 -> 주문**이라는 머신러닝 트레이딩의 기본 흐름이다.

읽다가 길을 잃었을 때 보는 지도

이 책에는 HMM, Kalman, Kelly, SAC, MLflow처럼 이름이 다른 도구가 많이 나온다. 처음 읽을 때는 도구 이름보다 **각 도구가 답하는 질문**을 먼저 보면 된다.

질문	도구	쉬운 역할
시장 기록이 믿을 만한가?	데이터 파이프라인	깨끗한 재료 준비
모델이 읽을 숫자는 무엇인가?	피처 엔지니어링	원자료를 질문으로 바꾸기
숫자가 너무 흔들리지는 않는가?	Kalman	노이즈 줄이기
지금 시장은 어떤 장인가?	HMM	시장 날씨 읽기
여러 모델 의견이 일치하는가?	Ensemble	회의실에서 의견 합치기
어떤 페어를 거래할까?	StatArb / Pairs	거래 후보 고르기
지금 얼마나 벌어졌나?	Z-score	고무줄 늘어남 재기
얼마나 크게 들어갈까?	Kelly / SAC	포지션 크기 정하기
이 거래를 버틸 수 있는가?	Risk	손실 한도 확인
주문이 실제로 처리됐는가?	Broker	체결과 포지션 확인
운영 중 계속 정상인가?	Monitoring / MLflow	성과와 모델 이력 감시

이 표를 계속 옆에 두고 읽으면 된다. 어려운 장이 나와도"지금 새 알고리즘을 외우는 중"이라고 생각하지 말고, "이 장은 위 질문 중 어느 것을 더 잘 답하려고 하는가"를 찾으면 된다.

또 하나의 기준은 **입력과 출력**이다. 어떤 모델이나 모듈이 나오면 먼저 아래 세 줄을 채워보자.

```
입력: 무엇을 받는가?
출력: 무엇을 돌려주는가?
다음 단계: 그 출력이 어디로 가는가?
```

예를 들어 HMM은 이렇게 정리된다.

```
입력: 수익률, 변동성, 모멘텀, 거래량 같은 피처
출력: MR / Trend / Volatile 레짐 확률
다음 단계: Z-score 해석, SAC 상태 벡터, 리스크 판단에 사용
```

SAC는 이렇게 정리된다.

```
입력: HMM 레짐 확률, Z-score, 현재 포지션, 손익, 변동성
출력: 환경 계약에 따라 ENTER/HOLD/EXIT 후보 또는 -1~+1 목표 포지션
다음 단계: 행동 후처리와 Risk 레이어가 의미·한도를 확인한 뒤 주문 여부 결정
```

이렇게 입력과 출력을 잡으면, 수식이나 코드가 조금 어려워도 전체 흐름을 잃지 않는다.

하루를 따라가 보는 전체 예시

개념이 추상적으로 느껴지면 하루짜리 장면으로 생각하면 쉽다. 예를 들어 2026년 5월 20일 장 마감 후 시스템이 다음 날 판단을 준비한다고 하자.

```
1. 데이터 수집
   오늘까지의 가격, 거래량, 금리, VIX를 가져온다.

2. 피처 생성
   원자료를 수익률, 변동성, 모멘텀, Z-score 같은 숫자로 바꾼다.

3. HMM 추론
   학습된 HMM 모델 파일을 불러와 오늘 시장 레짐 확률을 계산한다.
   예: MR 65%, Trend 20%, Volatile 15%

4. 페어 신호 확인
   두 종목의 가격 차이가 얼마나 벌어졌는지 Z-score로 본다.
   예: z-score = 2.4

5. RL/SAC 추론
   HMM 확률, Z-score, 현재 포지션, 손익 상태를 함께 넣는다.
   SAC가 "0.35만큼 진입" 같은 포지션 크기를 돌려준다.

6. 리스크 점검
   포지션 한도, 손실 한도, 거래 비용, 유동성을 확인한다.
```

```
7. 주문 또는 대기
   조건을 통과하면 주문을 내고, 아니면 HOLD로 끝낸다.
```

이 예시에서 모델이 새로 공부하는 단계는 없다. 장중이나 실시간 추론 단계에서는 이미 학습된 모델 파일을 불러와 판단만 한다. 다시 말해 운영 중인 모델은 매 순간"학습하는 학생"이 아니라"시험장에서 답안을 내는 계산기"에 가깝다.

반대로 학습은 별도의 시간에 한다. 예를 들어 주말이나 장 마감 후에 과거 데이터를 모아 HMM과 SAC를 다시 학습하고, 성능이 OOS에서 좋아졌을 때만 새 모델 파일을 등록한다. 그래서 실전 시스템은 아래처럼 두 개의 시간표로 나뉜다.

```
느린 시간표: 학습 / 검증 / 모델 등록
과거 데이터 -> 학습 -> OOS 검증 -> 모델 파일 저장 -> 승인

빠른 시간표: 실시간 추론 / 주문
오늘 데이터 -> 피처 생성 -> 모델 파일 로드 -> 추론 -> 리스크 체크 -> 주문 -> 체결 확인
-> 모니터링
```

이 둘을 섞지 않는 것이 중요하다. 느린 시간표는 모델을 만드는 공장이고, 빠른 시간표는 만들어진 모델을 사용하는 매장이다. 공장에서 레시피를 계속 바꾸는 것은 가능하지만, 손님에게 음식을 내는 순간에는 이미 정해진 레시피로 빠르고 일관되게 만들어야 한다.

여기에 하나를 더 붙이면 운영 시스템의 그림이 완성된다. 주문은 끝이 아니다. 주문이 브로커에 접수됐는지, 실제로 체결됐는지, 내부 포지션과 브로커 포지션이 일치하는지, 이후 성과가 기대 범위 안에 있는지 계속 확인해야 한다.

```
모델 관점:
오늘 상태 -> 판단 결과

운영 관점:
판단 결과 -> 주문 -> 체결 -> 포지션 -> 손익 -> 알림
```

그래서 이 책의 후반부에서 리스크 관리, IBKR 연결, 모니터링을 다룬다. 머신러닝 모델이 좋은 판단을 해도, 주문이 실패하거나 체결 확인이 누락되거나 성과 저하를 늦게 발견하면 실전 시스템은 무너진다. **주식시장을 파헤치는 일은 모델을 만드는 데서 끝나지 않고, 그 모델이 실제 돈 앞에서 계속 정상 작동하는지 확인하는 데까지 이어진다.**

모델을 더 쉽게 쪼개면

모델이라는 단어가 아직 모호하면 세 부분으로 나누어 보면 된다.

```
모델 = 입력 형식 + 학습된 숫자 + 출력 의미
```

HMM 모델을 예로 들면 입력 형식은 [수익률, 변동성, 모멘텀, 거래량 비율]이다. 학습된 숫자는 전이행렬 A, 상태별 평균과 분산, 초기 확률이다. 출력 의미는 [MR 확률, Trend 확률, Volatile 확률]이다.

SAC 모델을 예로 들면 입력 형식은 28차원 상태 벡터다. 학습된 숫자는 신경망 가중치다. 출력 의미는 포지션 크기다.

모델	입력	학습된 숫자	출력
HMM	시장 피처	A, 상태별 분포, π	레짐 확률
SAC Actor	RL 상태 벡터	신경망 가중치	포지션 크기
Z-score 규칙	스프레드 통계	평균과 표준편차	진입 후보

따라서"모델 파일을 저장한다"는 말은 단순히 파일 하나를 저장한다는 뜻이 아니다. 나중에 같은 입력을 넣었을 때 같은 의미의 출력을 얻기 위해, 입력 순서와 학습된 숫자와 출력 해석을 함께 고정한다는 뜻이다.

르네상스 사례에서 얻은 원칙

이 책 전체를 관통하는 원칙이 세 가지 있다. 르네상스 테크놀로지스의 트레이딩 방식을 분석하면서 뽑아낸 것들이다.

1. 인과성 원칙

> 미래 데이터는 현재 결정에 쓰면 안 된다.

당연하게 들리지만, 코드로 구현하다 보면 생각보다 쉽게 어긴다. 흔히 쓰는 라이브러리 함수 중에도 내부에서 미래 정보를 살짝 섞어 쓰는 것들이 있다. 이 책에서는 그런 함수 대신, "지금까지의 데이터만 본다"는 약속을 지키는 방식(Forward Algorithm)을 직접 구현한다. 자세한 이유는 5장에서 설명한다. 한 번 이해하고 나면 이후 코드에서 같은 문제가 바로 눈에 들어온다.

2. 앙상블 원칙

> 모델 하나에만 의존하지 않는다.

이 시스템은 일봉 HMM, 시간봉 HMM, 강화학습 에이전트, 통계적 차익거래 신호를 함께 사용한다. 모델의 판단이 엇갈리면 전략에 따라 관망하거나 포지션을 줄인다. 모든 모델의 만장일치를 항상 요구하는 것은 아니다. 어떤 출력은 진입 게이트로, 어떤 출력은 가중치나 포지션 배율로 쓰이므로 결합 규칙을 먼저 확인해야 한다.

3. Walk-forward 검증 원칙

> 전략이 실제로 작동하는지 확인하는 방법은 하나뿐이다.

모델이 한 번도 보지 못한 데이터로, 시간 순서대로 반복 테스트하는 것이다. 데이터를 한 번 나눠 훈련하고 테스트하는 방식만으로는 부족하다. Walk-forward 검증은 이 책 모든 전략에 적용하는 기본 기준이다.

앞으로의 여정

전체 시스템은 데이터 -> 노이즈 제거 -> 국면 감지 -> 포지션 결정 -> 검증과 실전 운영의 순서로 쌓아 올린다. 각 단계가 하나의 챕터 묶음으로 이어진다.

단계	하는 일
1	시장 데이터를 수집하고 저장한다
2	가격에 섞인 잡음을 줄인다

단계	하는 일
3	여러 자산을 함께 보는 HMM으로 시장 국면을 추정한다
4	거시/미시 두 단계로 국면을 Risk-On / Neutral / Risk-Off로 정리한다
5	국면에 맞춰 얼마나 큰 포지션을 잡을지 정한다
6	통계적 차익거래로 추가 수익 기회를 찾는다
7	강화학습 에이전트가 포지션 판단을 보조한다
8	리스크 관리, 브로커 연결, 모니터링으로 운영 조건을 갖춘다

처음 읽을 때는 0장에서 실행 경험을 만든 뒤 1~18장을 **기본 시스템을 한 번 완성하는 흐름**으로 보면 된다. 19~27장은 같은 재료를 더 깊게 다루는 심화부다. 페어 이론, 완전한 HMM 파이프라인, 운영 검증, 분포형 RL, MLflow 서빙처럼 실제 운용 단계에서 다시 필요해지는 내용을 모아 둔 구간이다.

참고: 백테스트 성과를 읽는 법

이 책에는 여러 백테스트 결과가 등장하지만, 특정 숫자를 성과 약속처럼 읽으면 안 된다. 같은 전략도 데이터 기간, 거래 비용, 종목군(universe), 재학습 주기, 슬리피지 가정에 따라 결과가 크게 달라진다.

첫 회독에서는 숫자보다 다음 세 가지를 본다.

- 미래 데이터를 보지 않았는가
- 최대 낙폭과 거래 비용을 함께 봤는가
- 표본 밖 구간에서도 비슷한 패턴이 유지되는가

전체 챕터 로드맵

Part	챕터	주제
시작	머리말·0장	독자 수준 확인, AI와 첫 백테스트 실행
I	1-3장	시스템 전체 그림, Look-ahead Bias, 데이터 파이프라인, 백테스트 엔진
II	4-5장	HMM 기본 구조와 학습 알고리즘, 미래 정보 없이 추론하는 법
III	**6-8장**	칼만 필터(노이즈 제거), 여러 자산을 함께 보는 HMM, 2단계 거시/미시 HMM (시스템의 핵심)
IV	9-10장	포지션 크기 정하기, HMM 앙상블로 방향 신호 만들기
V	11-12장	함께 움직이는 페어 찾기와 통계적 차익거래 신호 만들기
VI	13-15장	강화학습 환경, SAC 에이전트, 하이퍼파라미터 튜닝
VII	16-18장	리스크 관리, 브로커 연동, 실시간 모니터링
VIII	19-27장	심화: 페어 이론, 레짐 기반 Z-score, 완전 파이프라인, 운영 검증, 분포형 RL, MLflow 서빙

> 표에 등장한 용어 대부분은 처음 보면 낯설다. 지금 외울 필요는 없다. 첫 회독에서는 Part I~VII로 "데이터에서 주문과 모니터링까지"한 바퀴를 돈다. Part VIII은 두 번째 회독 또는 실제 시스템을 다듬을 때 필요한 심화 구간으로 보면 된다.

회독별로 다르게 읽기

이 책은 한 번에 모든 수식과 코드를 이해하도록 쓴 책이 아니다. 처음부터 모든 파라미터와 구현 세부를 잡으려고 하면 흐름을 놓치기 쉽다. 회독마다 목표를 다르게 잡는 편이 낫다.

회독	목표	읽는 기준
1회독	큰 흐름 잡기	데이터가 어떻게 모델 판단과 주문으로 이어지는지 본다
2회독	코드와 개념 연결	각 장의 입력, 출력, 위험 지점을 코드에서 확인한다
3회독	실험과 수정	파라미터를 바꾸고 OOS 결과가 어떻게 달라지는지 본다

바이브코더라면 회독보다 **작동 경로**를 먼저 잡아도 된다. `0장 -> 1~3장 -> 11~12장 -> 16~18장` 순서로 읽으면 데이터, 백테스트, 단순한 페어 신호, 리스크, 주문, 모니터링까지 한 바퀴를 먼저 볼 수 있다. 4~10장의 HMM과 13~15장의 강화학습 내부는 그다음 회독에서 돌아온다. 건너뛴 장이 덜 중요하다는 뜻이 아니라, 추상적인 모델을 보기 전에 시스템의 입출력을 먼저 경험하라는 뜻이다.

1회독에서는 수식을 완전히 이해하지 못해도 괜찮다. 대신 각 장이 아래 질문 중 무엇을 다루는지 잡으면 된다.

> 이 장은 데이터를 준비하는 장인가?
> 시장을 읽는 장인가?
> 기회를 찾는 장인가?
> 크기를 정하는 장인가?
> 위험을 막는 장인가?
> 운영 상태를 확인하는 장인가?

2회독에서는 코드로 내려간다. `fit()`은 어디서 호출되는지, `predict()`나 `step_forward()`는 어느 구간에서만 실행되는지, OOS(out-of-sample, 표본 외) 구간이 학습 구간과 섞이지 않았는지 확인한다. 이때부터 수식은"외울 대상"이 아니라"코드가 왜 이렇게 생겼는지 설명하는 지도"가 된다.

3회독에서는 실험한다. 윈도우 길이, 진입 임계값, 레짐 수, 거래 비용, 리스크 한도 같은 값을 조금씩 바꿔본다. 단, 같은 OOS 결과를 보면서 계속 튜닝하면 안 된다. 값을 바꿨다면 어떤 가설로 바꿨는지 기록하고, 검증 구간과 최종 테스트 구간을 분리해야 한다.

Part별 최소 이해 목표

각 Part에서 처음 읽을 때 반드시 잡아야 할 한 문장은 다음과 같다.

Part	최소 이해 목표
I	데이터와 백테스트가 거짓말하지 않게 기반을 만든다
II	HMM은 가격이 아니라 시장 상태를 확률로 읽는다
III	노이즈를 줄이고 여러 시장을 함께 봐야 레짐 판단이 안정된다
IV	신호는 방향이고, Kelly는 그 방향을 돈의 크기로 바꾼다
V	페어 트레이딩은 종목 방향이 아니라 두 가격의 관계를 거래한다
VI	RL(강화학습)은 새 신호를 만드는 만능 AI가 아니라 포지션 크기를 조절하는 층이다
VII	모델이 맞아도 리스크, 주문, 모니터링이 틀리면 실전 시스템은 실패한다
VIII	같은 재료를 더 엄격하게 검증하고 운영 가능한 형태로 다듬는다

첫 회독의 기준은 이 표 하나면 충분하다. 세부 수식이 어렵더라도 각 Part의 최소 목표를 자기 말로 설명할 수 있다면 그 회독은 성공이다.

AI를 페어 프로그래머로 쓰기

이 책은 막히는 부분마다 AI 코딩 어시스턴트를 어떻게 활용했는지 직접 보여준다. 알고리즘을 이해할 때 쓴 프롬프트, 어시스턴트가 찾아낸 미묘한 버그, 그 대화 덕분에 더 나은 방향으로 바꾼 순간들이 본문 곳곳에 담겨 있다.

목표는 생각을 기계에 넘기는 게 아니다. **좋은 시니어 동료를 대하듯** 쓰면 된다. 모르는 건 묻고, 이해한 건 확인하고, 아는 부분은 빠르게 넘어가고, 막힌 부분은 함께 풀어간다.

각 장의 프롬프트는 네 요소를 반복한다.

```
목표: 이번에 얻을 한 가지 결과
범위: 읽거나 수정해도 되는 파일
금지: 실주문, 미래 데이터 사용, 근거 없는 파일명 추측
검증: 어떤 명령과 출력으로 완료를 확인할지
```

AI가 만든 코드는 실행 결과와 테스트로 확인한다. 특히 트레이딩에서는"코드가 실행된다"와"검증이 정직하다"가 전혀 다른 조건이다. 미래 데이터가 섞이지 않았는지, 거래 비용이 반영됐는지, 실주문으로 연결되지 않았는지를 사람이 마지막에 확인해야 한다.

퀀트 트레이딩이 박사들만의 영역은 아니다. 도구와 방향을 제대로 잡으면 실용적인 트레이딩 시스템을 직접 만들 수 있다. 이 책은 그 과정을 기록한 것이다.

이 장에서 반드시 기억할 것

- 이 책은 차트 패턴을 더 잘 외우는 책이 아니다. 시장을"데이터로 보는 시스템"을 만드는 책이다.
- 가장 중요한 약속은 단 하나, **"미래 데이터를 현재 결정에 쓰지 않는다"**이다. 이 약속이 깨지면 좋아 보이는 수치도 정직한 검증 결과로 믿을 수 없다.
- 모델 하나에 모든 걸 걸지 않는다. 모델 합의도, 가중 평균도, 위험할 때의 관망도 사전에 정한 결합 규칙과 OOS 검증을 거쳐 사용한다.
- 처음부터 모든 수학을 완벽히 알 필요는 없다. 각 장에서"왜 이 개념이 필요한가", "잘못 만들면 어떤 문제가 생기는가"를 순서대로 따라가면 된다.

사전 요구사항

본격적인 내용에 들어가기 전, 다음 정도는 준비되어 있어야 한다.

- **Python 3.12 이상** —표준 라이브러리 실습은 본문만으로 실행한다. 수치 계산·RL·웹 실습은 각 장에 적은 공개 패키지를 설치한다.

준비됐다면, 다음 장에서 전체 시스템의 설계도를 펼쳐 보자.

0장. 바이브코딩 퀵스타트—먼저 한 번 검증해보기

이 장의 목표는 큰 자동매매 시스템을 설치하는 것이 아니다. 본문의 작은 숫자 예를 따라가며 **AI가 설명한 손익이 맞는지 직접 확인하는 것**이다. 운영 저장소·브로커 계정·시장 데이터 없이 읽을 수 있다. Python을 실행할 수 있다면 아래 코드까지 확인하고, 실행 환경이 없으면 표를 손으로 계산한다.

먼저 답을 정한다

초기 현금 1,000으로 가격 100인 자산 10개를 보유한다고 하자. 이후 가격은 80으로 떨어졌다가 100으로 돌아온다. 비용은 이 첫 계산에서 0으로 둔다.

시점	현금	보유 수량	가격	총자산
거래 전	1,000	0	100	1,000
매수 후	0	10	100	1,000

시점	현금	보유 수량	가격	총자산
가격 하락	0	10	80	800
가격 회복	0	10	100	1,000
청산 후	1,000	0	100	1,000

총자산은 현금 + 수량 × 가격이다. 최종 손익은 0이지만 가장 큰 낙폭은 `(1000-800)/1000=0.2`, 즉 20%다. 최종 잔고만 보면 중간에 감수한 위험을 놓친다.

본문만으로 실행하는 확인 코드

다음을 `first_check.py`라는 새 파일에 붙인다. Python 표준 라이브러리만 사용하며 파일 다운로드가 필요 없다.

```
prices = [100.0, 80.0, 100.0]
initial = 1000.0
quantity = initial / prices[0]
cash = initial - quantity * prices[0]
equity = [initial] + [cash + quantity * p for p in prices]
peak = initial
mdd = 0.0
for value in equity:
    peak = max(peak, value)
    mdd = max(mdd, (peak - value) / peak)
final_cash = cash + quantity * prices[-1]
assert final_cash == 1000.0
assert mdd == 0.2
print(final_cash, mdd)
```

터미널에서 `python3 first_check.py`를 실행하면 `1000.0 0.2`가 출력된다. Windows에서 Python 명령이 `py`라면 `py first_check.py`를 쓴다. 이 코드는 고정 수량의 손익 검산이다. 신호 생성·실제 주문·HMM 학습을 포함하지 않는다. 처음부터 끝까지 실행 가능한 비용 포함 엔진은 3장 본문에 있다.

AI에게 요청하는 방법

```
위 표와 Python 코드를 함께 검토해줘.
1. 총자산과 최대 낙폭을 손으로 계산해줘.
2. 현금만 기록하면 어떤 위험을 놓치는지 설명해줘.
3. 매수·매도에 각각 10bp 수수료가 붙으면 최종 현금을 계산해줘.
4. 브로커·외부 데이터·새 패키지를 추가하지 마.
답을 낸 뒤 표의 어느 값으로 검증했는지 표시해줘.
```

10bp는 거래 금액의 0.1%다. 수량 10을 100에 매수·매도하면 각 비용은 1, 합계는 2, 최종 현금은 998이다. 이 비용 예에서는 초기 전액을 수량으로 정해 수수료 1만큼 현금이 잠시 음수가 된다. 차입을 허용하지 않는 실제 계좌라면 수수료를 남기도록 매수 수량을 줄여야 한다. 3장의 고정 수량 실습과 실제 계좌 제약을 구분한다.

시간 순서를 확인한다

신호가 오늘 종가를 보고 만들어졌다면 그보다 앞선 오늘 시가로 체결했다고 가정할 수 없다. 이 책의 기본 실습은 **t봉 종가 뒤에 신호 확정 -> t+1봉 시가 체결 -> t+1봉 종가 평가**다. 앞서 확정된 신호로 시가에 진입한 포지션의 당일 손익은 정상적으로 포함한다.

실패를 좁히는 순서

증상	먼저 확인할 것
Python 명령을 찾지 못함	설치된 명령이 python3 또는 py인지 확인
들여쓰기 오류	코드 블록만 복사했는지와 들여쓰기 확인
낙폭이 0으로 나옴	가격 80의 자산 800을 기록했는지 확인
최종 현금과 손익 합계 불일치	누적 손익을 다시 더했는지·비용이 중복됐는지 확인

새 패키지를 설치해 오류를 덮지 말고 위 코드와 다른 줄부터 확인한다. 좋은 답을 생성했는지보다 기대값과 실제 출력이 맞는지를 우선한다.

다음 장으로 넘어갈 기준

- 최종 손익 0과 최대 낙폭 20%가 동시에 성립하는 이유를 설명한다.
- 수수료를 실제 체결마다 한 번씩 계산한다.
- 신호 생성 시각과 체결 시각을 구별한다.

이 세 가지를 확인하면 1장의 구조와 3장의 확장 실습으로 넘어간다. 별도의 예제 파일은 제공하지 않으므로 실습 코드는 본문에서 직접 저장해 사용한다. 책 속 `app.trading` 같은 경로는 저자의 시스템 구조를 설명하는 사례 이름이며, 그 비공개 패키지가 있어야 본문의 개념을 이해하는 전제는 두지 않는다.

전체 책에서 계속 읽기

짧은 미리보기는 여기까지입니다. 전체 책에는 다음 장이 포함되어 있습니다.

- 머리말
- 0장. 바이브코딩 퀵스타트—먼저 한 번 검증해보기
- 1장. 시스템 아키텍처—AI 퀀트 트레이딩의 설계 철학
- 2장. 데이터 파이프라인—DuckDB와 실시간 피드
- 3장. 백테스트 엔진—신뢰할 수 있는 시뮬레이션
- 4장. HMM 기초—시장 레짐을 확률로 추론하기
- 5장. HMM Forward-Only —미래 정보 없는 실시간 추론
- 6장. 칼만 필터—노이즈 제거와 혁신 Z-스코어
- 7장. 크로스애셋 Joint HMM —여러 시장을 함께 보는 레짐 탐지
- 8장. 2계층 HMM 앙상블—Layer A × Layer B
- 9장. 켈리 절반 포지션 사이징
- 10장. HMM 포워드 앙상블—V1 -> V3 진화
- 11장. 통계적 차익거래—페어 선정
- 12장. 통계적 차익거래—신호 생성
- 13장. RL 환경 설계—StatPairRLEnv
- 14장. SAC 에이전트—아키텍처와 훈련

- 15장. Optuna 하이퍼파라미터 튜닝
- 16장. 리스크 관리—포지션 보호와 리스크 지표
- 17장. IBKR 연결—실시간 주문 체결
- 18장. 모니터링—실시간 성과 추적과 알림
- 19장. 뱀버거 통계적 페어 차익거래—이론과 원칙
- 20장. 페어 선별—레짐 조건부 필터링
- 21장. 레짐 기반 Z-스코어—핵심 전략 로직
- 22장. HMM 레짐 전략—데이터에서 신호까지 완전한 파이프라인
- 23장. SAC RL 에이전트 프리뷰—강화학습 입문
- 24장. 프로덕션 운영 검증
- 25장. RL 알고리즘 비교—SAC vs PPO vs QR-DQN
- 26장. 분포적 RL과 CVaR —꼬리 리스크 거래
- 27장. MLflow 모델 레지스트리와 서빙—프로덕션 배포
- 부록. 초보자를 위한 전체 지도

전체 책 구매